Relatório: LLM x RAG — Testes e Métricas

Nov 11, 2026 · @Ian

Resumo

O RAG sem LLM foi mais exato que qualquer configuração com LLM, e a troca do modelo de embeddings foi a melhoria que mais rendeu. Comparamos quatro formas de responder (LLM pura, RAG sem LLM, RAG + LLM e RAG otimizado + LLM) em 27 perguntas em linguagem popular.

- **LLM pura** inventou todas as leis que citou e recusou 71% das perguntas válidas.
- **RAG sem LLM** acertou programa e lei em 90%, mas responde até "bolo de cenoura" com programas sociais.
- **RAG + LLM** passou a recusar o que é fora do escopo, mas o modelo de 3B perde o programa ou a lei em boa parte das respostas e leva 5 a 7 s.
- **Embeddings multilíngues (bge-m3)** levaram a busca de 90% para 100% no top-4 e permitiram recusar pelo score.

Escolha para o produto: busca bge-m3 + recusa por limiar + resposta por modelo de texto fixo (T5): 100% de programa e lei corretos, 100% de recusa correta e 0,2 s por resposta.

Contexto do projeto e links

O Assistente Virtual Tira-Dúvidas (Grupo 7) responde perguntas em linguagem popular sobre 42 programas sociais federais. Os testes rodaram na branch `ConsolidacaoRagLLM`, sobre o protótipo da pasta `rag/`.

Item	Link
Repositório	purpleaither/SegundoChallenge
Branch dos testes	ConsolidacaoRagLLM
Protótipo RAG (ingestão, busca, avaliação)	rag/
Script dos experimentos	rag/experimentos.py
Resultados brutos (JSON por pergunta)	rag/resultados/
Base curada (CSV)	programas_sociais_legislacao_links_oficiais.csv

O script e os resultados ainda não foram enviados ao GitHub; os links passam a funcionar depois do commit e push na branch.

Metodologia

As quatro configurações (e a combinação T5, montada depois a partir dos resultados) responderam as mesmas 27 perguntas, com o mesmo modelo de linguagem e as mesmas métricas automáticas. Tudo roda localmente, sem custo de API.

Configurações testadas

Teste	Recuperação	Geração da resposta
T1 — LLM pura	nenhuma	qwen2.5:3b só com o que aprendeu no treino
T2 — RAG sem LLM (protótipo da semana 2)	ChromaDB + all-MiniLM-L6-v2, top-4 programas	modelo de texto fixo preenchido com a ficha do programa
T3 — RAG + LLM	igual ao T2	qwen2.5:3b redige usando só as 4 fichas recuperadas, com citações [n]
T4 — RAG otimizado + LLM	ChromaDB + bge-m3 (multilíngue) e limiar de similaridade para abster	igual ao T3

Base e perguntas

- Base: 42 fichas de programas (uma por programa), quebradas em 126 trechos por seção (*O que é*, *Quem pode*, *Legislação*). Os mesmos trechos foram indexados com os dois modelos de embeddings.
- 21 perguntas dentro do escopo, em linguagem popular, cada uma com o programa esperado (as mesmas do `avaliar.py`).
- 6 perguntas fora do escopo (futebol, receita, previsão do tempo, filme, passaporte, celular), para medir se o sistema sabe dizer "não sei".
- Em todos os testes as perguntas passam pela mesma expansão de fala popular ("botijão" → "auxílio gás GLP").

Controle de variáveis

- LLM: qwen2.5:3b via Ollama, temperatura 0, seed 42, até 400 tokens. O mesmo prompt-base em T1, T3 e T4; em T3 e T4 ele recebe as fontes e a regra "use somente as fontes".

- Quando a pergunta não é sobre programas sociais, o prompt pede a resposta `FORA_DO_ESCOPO`, o que torna a abstenção mensurável.
- O limiar do T4 é o ponto médio entre o menor score das perguntas válidas e o maior score das perguntas fora do escopo.

Métricas

Métrica	O que mede	Como é calculada
Acerto@1, Acerto@4, MRR	qualidade da busca	posição do programa esperado entre os 4 recuperados
Margem de separação	se dá para abster pelo score	menor score dentro do escopo − maior score fora
Programa correto	a resposta aponta o programa certo	nome (ou apelido oficial) do programa esperado no texto
Lei correta	a base legal citada é a certa	número da lei do programa esperado aparece na resposta (35 das 42 fichas têm lei-base)
Norma sem lastro	alucinação de base legal	leis, decretos ou portarias citados que não existem em nenhum documento da base
Abstenção correta / indevida	se recusa o que deve e só o que deve	`FORA_DO_ESCOPO` nas 6 perguntas fora / nas 21 dentro
Latência	custo para o usuário	segundos por pergunta, num Mac com CPU/GPU local

Reprodução: `cd rag && .venv/bin/python experimentos.py`. As respostas completas de cada teste ficam em `rag/resultados/T*.json`.

Limitações: o conjunto é pequeno (27 perguntas) e o limiar do T4 foi calibrado nas mesmas perguntas, então tende a ser otimista. As métricas de texto conferem nomes e números, não a qualidade da redação; por isso a análise inclui leitura manual de respostas.

Resultados

Nenhuma das três configurações com LLM superou o RAG sem LLM em exatidão. O único ganho real do LLM foi saber recusar perguntas fora do escopo. A combinação T5 (busca e abstenção do T4 com a resposta por modelo de texto do T2) foi a melhor em todas as métricas.

Respostas (21 perguntas dentro do escopo, 6 fora)

Teste	Programa correto	Lei correta	Norma sem lastro	Abstenção correta (fora)	Abstenção indevida (dentro)	Latência média
T1 — LLM pura	10%	0%	29% das respostas (100% das normas citadas)	100%	71%	1,1 s
T2 — RAG sem LLM	90%	90%	0%	0%	0%	0,1 s
T3 — RAG + LLM	67%	5%	0%	100%	5%	7,2 s
T4 — RAG otimizado + LLM	76%	10%	0%	100%	0%	5,7 s
T5 — busca do T4 + texto fixo (escolhido)	**100%**	**100%**	**0%**	**100%**	**0%**	**0,2 s**

Recuperação (só a busca, 21 perguntas)

Busca	Acerto@1	Acerto@4	MRR	Menor score dentro	Maior score fora	Margem
MiniLM	33%	43%	0,36	0,46	0,56	−0,10
MiniLM + expansão (T2, T3)	76%	90%	0,83	0,56	0,56	0,00
bge-m3	81%	90%	0,86	0,50	0,49	+0,01
bge-m3 + expansão (T4, T5)	**86%**	**100%**	**0,92**	**0,57**	**0,49**	**+0,08**

Limiar de abstenção calibrado para T4 e T5: similaridade 0,532.

A mesma pergunta nos quatro testes: "minha mãe é idosa e não tem renda nenhuma"

- **T1:** "pode ser o Benefício do Idoso (BI). Ele foi criado pela Lei 12.595/2012." O programa não existe e a lei não tem relação com o tema.
- **T2:** lista o BPC com público, critérios e "Lei nº 8.742/1993 (LOAS); Lei nº 15.077/2024", seguido de outros 3 programas.
- **T3:** "O programa que pode te ajudar é o [1] Benefício de Prestação Continuada (BPC)…" Correto, mas sem a lei.
- **T4:** igual ao T3, mais claro ("para sua mãe"), também sem a lei.

O que muda e por que muda

A recuperação decide se a informação certa chega; o LLM decide o que sobra dela na resposta. Com um modelo de 3 bilhões de parâmetros, o LLM mais perde do que acrescenta.

Sem RAG (T1): o modelo não conhece a base e inventa.

- Todas as 6 normas que ele citou são falsas ou não têm relação com o programa (ex.: BPC atribuído à "Lei 8.083/90"; o correto é a Lei nº 8.742/1993).
- Inventou programas ("Benefício do Idoso").
- Recusou 15 das 21 perguntas válidas ("botijão de gás", "conta de luz"). Sem fontes, o modelo pequeno não reconhece fala popular como assunto de programa social e usa a saída de recusa como atalho.
- Por que: o conhecimento do modelo é genérico, desatualizado e sem números de lei confiáveis. É o cenário que o RAG existe para evitar.

Com RAG e sem LLM (T2): exato, mas não sabe dizer "não sei".

- O texto vem direto da ficha curada, então lei e critérios nunca são inventados (0% de norma sem lastro, 90% de lei correta).
- Para "bolo de cenoura" ele também devolve 4 programas: com o MiniLM, o score de perguntas fora do escopo (até 0,56) empata com o de perguntas válidas, então não existe limiar que separe as duas.
- A resposta é longa: sempre lista 4 programas, mesmo quando só um interessa.

Com RAG e com LLM (T3, T4): mais natural, menos completo.

- O LLM aprendeu a recusar bem: 100% das perguntas fora do escopo e quase nenhuma recusa indevida, porque agora ele vê se as fontes têm relação com a pergunta.
- As fontes eliminaram as leis inventadas (0% sem lastro), mas o modelo quase nunca copia a lei para a resposta (5% e 10%), mesmo com a instrução explícita no prompt.
- No T4 a busca entregou o programa certo em 100% dos casos, mas a resposta só o nomeou em 76%. Os erros são do redator: escolheu a fonte errada (Pé-de-Meia virou PNAES, que é para faculdade), omitiu o nome do programa ("Pode, mas dependendo da situação…") e uma vez gerou texto corrompido (caracteres chineses repetidos).
- Custo: 5 a 7 s por resposta, contra 0,2 s sem LLM.

Trocar o embedding (T3 → T4) foi a melhoria que mais rendeu.

- O all-MiniLM-L6-v2 foi treinado quase só em inglês; sem a expansão de sinônimos ele acerta 43% no top-4. O bge-m3 é multilíngue e acerta 90% sem ajuda e 100% com a expansão.

- O bge-m3 também abriu uma margem de 0,08 entre perguntas válidas e fora do escopo. É essa margem que permite abster só pelo score, sem gastar uma chamada ao LLM.

Por que o T5 ganha: ele junta a parte forte de cada teste. A busca bge-m3 traz o programa certo e decide quando abster; o modelo de texto fixo copia lei e critérios da ficha sem chance de distorcer. O LLM só acrescentaria fluidez, e com o qwen2.5:3b essa fluidez custa exatidão.

Decisão para o produto

Escolhemos o **T5: busca com bge-m3 + expansão de fala popular, abstenção por limiar de similaridade e resposta por modelo de texto fixo, sem LLM.** Ele acertou programa e lei em 100% das perguntas, recusou as 6 fora do escopo e responde em 0,2 s, sem custo de API.

Entre os quatro testes pedidos, o melhor equilíbrio com LLM foi o T4; o T5 nasceu da análise dos quatro e foi validado com as mesmas métricas.

Próximos passos

- [ ] Levar a busca com bge-m3 e o limiar 0,53 para o `buscar.py` e a API.
- [ ] Ampliar o conjunto de teste (≥ 50 perguntas, ≥ 15 fora do escopo) e recalibrar o limiar em perguntas separadas das usadas na calibração.
- [ ] Encurtar a resposta: mostrar o primeiro programa por completo e os demais só pelo nome.
- [ ] Reavaliar o LLM apenas como redator opcional, com um modelo maior (7B ou mais) e checagem automática de que programa e lei da resposta estão nas fontes.
- [ ] Corrigir o caminho do CSV em `rag/ingestao.py`, que aponta para a raiz do repositório, mas o arquivo está em `Base de Dados e RAG /`. Corrigido em 11/11/2026; a reingestão gera fichas idênticas às de `rag/base/`.